TARGET PERIPHERAL: Glorious Model (Pixart Architecture)

PROJECT DESIGNATION: `gloriousctl-linux`

CLASSIFICATION: Hardware Reverse Engineering & Driver Synthesis Lab Report

PHASE I: OPERATIONAL BRIEF & RECONNAISSANCE

The objective of this operation was to develop a native, open-source Linux configuration utility for newer Glorious mice. enkore's `gloriousctl` utility (made roughly six years ago) was deemed outdated, as the manufacturer transitioned away from the original Sinowealth microcontrollers to a new Pixart architecture. To establish a baseline without relying on a Windows Virtual Machine for daily configuration, the proprietary communication protocol had to be decapitated and rebuilt from the ground up.

The lab environment was constructed using a Type-2 Hypervisor (VirtualBox) running a Windows 11 guest. The peripheral was passed through to the guest OS, allowing the proprietary Glorious Core software to interface with the hardware, while the Linux host maintained a packet sniffing connection for the raw USB bus. Feature isolation was strictly enforced: theme, DPI, and performance settings were toggled one at a time to ensure traffic captures were clean and untainted by overlapping commands.

PHASE II: INTERCEPTION VECTORS

Host-side telemetry was established using `modprobe usbmon`, `lsblk`, and `tshark`. Rather than analyzing raw, unfiltered data streams, specific interception rules were deployed to isolate the exact Host-to-Device (H2D) and Device-to-Host (D2H) handshake sequences.

The following capture filters were strictly utilized to isolate the configuration payloads:

- `tshark -i usbmon3 -Y "frame.len == 128 && usb.bmRequestType == 0x21" -T fields -e usb.data_fragment`
- `tshark -i usbmon3 -Y "frame.len == 128" -T fields -e usb.bmRequestType -e usb.data_fragment > startup_handshake.txt`
- `tshark -i usbmon3 -Y "usb.bmRequestType == 0xa1" -T fields -e usb.data_fragment > read_check.txt`

PHASE III: CRYPTANALYSIS & PAYLOAD DECONSTRUCTION

With the captures secured, a custom Python diffing parser was engineered to perform byte-by-byte comparisons. This revealed that the manufacturer had significantly expanded the configuration payload. Initially presumed to be a 192-byte sequence, the startup handshake confirmed it to be a 256-byte payload. Because of the strict limitations of the USB HID protocol, this payload was being sliced into four separate 64-byte fragments before transmission.

During the decoding of the DPI memory registers, a clever hardware-level compression trick was uncovered. Modern mice support extreme sensitivities (up to 26,000 DPI), which exceeds the boundaries of standard byte allocation if stored linearly. The packet analysis revealed a static mathematical division: *Target DPI / 50*. By dividing 26,000 by 50, the maximum stored value becomes 520, allowing the entire DPI spectrum to neatly fit into a small, highly efficient byte sequence within the packet fragment.

PHASE IV: THE HARDWARE PANIC & BYPASS

Initial deployment of the reconstructed C codebase resulted in a failure. The legacy blueprint assumed standard Read/Write privileges over the USB interface. However, when the custom driver attempted to query the mouse state via `hid_get_feature_report`, the hardware returned null zeroes, entered a panic state, and immediately wiped itself to factory defaults.

This behavior confirmed a cost-saving measure by the manufacturer: the new Pixart implementation is strictly **Write-Only**. The physical hardware possesses no instruction set to report its current configuration back to the host computer.

To bypass this hardware limitation, a Local Linux Cache (`~/.gloriousctl_state.bin`) was constructed. This file acts as a virtual brain, storing the physical state of the mouse locally. The software reads and manipulates this local binary file, and then executes a blind push of the final payload to the peripheral, entirely circumventing the need for two-way communication.

PHASE V: FIRMWARE SYNTHESIS & THE BLACKOUT BUG

Translating the Python reconnaissance into a stable C binary required aggressive memory management. The C compiler's natural padding behaviors corrupted the byte alignment required by the USB packets. This was neutralized by wrapping the payload structures in `#pragma pack(push, 1)`, forcing the compiler to respect the exact byte dimensions dictated by the hardware.

The final anomaly encountered was the "Blackout Bug." When pushing multi-packet RGB animations (such as the "Rave" effect), the mouse LEDs would abruptly power down. Packet tracing revealed that the firmware requires the specific *Effect ID* to be perfectly synchronized across all three of the fragmented lighting packets. If Fragment 1 dictates an animation, but Fragments 2 and 3 default to zero, the microcontroller aborts the sequence to protect itself. Implementing a state-sync across all fragment headers permanently resolved the anomaly, resulting in operational hardware control.

APPENDIX: TACTICAL COMMAND REFERENCE

The following log details the exact toolchain and terminal commands utilized during the interception, analysis, compilation, and deployment phases of this operation.

1. Environment Initialization & Kernel Probing To intercept USB traffic natively on Linux, the `usbmon` kernel module was loaded and permissions were granted for packet sniffing.

Bash
sudo modprobe usbmon
sudo chmod 644 /dev/usbmon* (lazy way?)
sudo apt install tshark

2. Traffic Interception & Packet Sniffing `tshark` was deployed to isolate specific Host-to-Device (H2D) and Device-to-Host (D2H) payloads, stripping away standard USB headers to capture pure hex data.

Bash
# Intercepting live configuration changes (Filtering for 128-byte frame length and H2D requests)
sudo tshark -i usbmon3 -Y "frame.len == 128 && usb.bmRequestType == 0x21" -T fields -e usb.data_fragment

Capturing the initial startup handshake upon software launch
tshark -i usbmon3 -Y "frame.len == 128" -T fields -e usb.bmRequestType -e usb.data_fragment > startup_handshake.txt

Verifying Write-Only status by listening for isolated D2H control transfers
tshark -i usbmon3 -Y "usb.bmRequestType == 0xa1" -T fields -e usb.data_fragment > read_check.txt

3. Cryptanalysis & Parsing A custom Python script was engineered to execute byte-by-byte diffing across the intercepted payload fragments.

Bash
vim pixart_parse.py
chmod +x pixart_parse.py
./pixart_parse.py data_fragment_settings.txt | cat -s > data_fragment_settings_conv.txt

4. Driver Compilation & Hardware Testing The C firmware was iteratively compiled and tested against the physical hardware.

Bash

```
make
# Testing the local Linux cache generation
sudo ./gloriousctl --info

# Pushing performance parameters (Debounce and DPI)
sudo ./gloriousctl --set-debounce-time 4 --set-dpi 400,800,1000,2000

# Pushing complex RGB payload arrays
sudo ./gloriousctl --set-effect rave --set-colors FF0000,0000FF --set-brightness 4 --set-speed 3

# Global installation to the system path
sudo cp gloriousctl /usr/local/bin/
```